

Sprint 2 — Documentação de Integração com MOM

1. Documentação dos Eventos

Nesta sprint foram implementados dois eventos de domínio relacionados a agendamentos. A tabela abaixo descreve cada um deles:

Tabela de Eventos

Campo	<code>appointment.created</code>	<code>appointment.status_changed</code>
Nome do evento	<code>appointment.created</code>	<code>appointment.status_changed</code>
Routing Key	<code>appointment.created</code>	<code>appointment.status_changed</code>
Exchange	<code>pointdog.events</code> (type: topic)	<code>pointdog.events</code> (type: topic)
Fila	<code>pointdog.notifications</code>	<code>pointdog.notifications</code>
Binding pattern	<code>appointment.*</code>	<code>appointment.*</code>
Produtor	<code>CreateAppointmentUseCase</code> (src/modules/appointments/application/create-appointment.use-case.ts)	<code>UpdateAppointmentStatusUseCase</code> (src/modules/appointments/application/update-appointment-status.use-case.ts)
Consumidor	<code>handleAppointmentEvent</code> (src/workers/appointment.worker.ts)	<code>handleAppointmentEvent</code> (src/workers/appointment.worker.ts)
Quando é publicado	Quando um agendamento é criado com sucesso; o status inicial é sempre PENDENTE	Quando o status de um agendamento muda: PENDENTE→CONFIRMADO , PENDENTE→CANCELADO , CONFIRMADO→CONCLUIDO

Campo	appointment.created	appointment.status_changed
		ou CONFIRMADO→CANCELADO
Durabilidade	Mensagem persistente, fila durável	Mensagem persistente, fila durável

Payload de exemplo: **appointment.created**

Abaixo está um exemplo real gerado durante os testes (2026-05-24):

```
{
  "eventType": "appointment.created",
  "timestamp": "2026-05-24T23:08:22.231Z",
  "payload": {
    "appointmentId": "982aa394-ad4d-4578-b2ee-1733cd057c7b",
    "clientId": "69636b0b-5aab-43ad-b15c-66d935e961b5",
    "petId": "630ed27c-5d6b-4d4d-8f18-3a153da18ddf",
    "providerId": "677541c2-f981-49e0-a68a-115a74e47284",
    "serviceId": "af7f74e3-1666-44b4-acdb-979f7f0b1bf4",
    "scheduledAt": "2027-03-10T14:00:00.000Z",
    "status": "PENDENTE"
  }
}
```

Payload de exemplo: **appointment.status_changed**

Abaixo está um exemplo real gerado durante os testes (2026-05-24):

```
{  
  
  "eventType": "appointment.status_changed",  
  
  "timestamp": "2026-05-24T23:08:44.115Z",  
  
  "payload": {  
  
    "appointmentId": "316006f2-a675-4f87-b390-614a93554ebd",  
  
    "clientId": "69636b0b-5aab-43ad-b15c-66d935e961b5",  
  
    "providerId": "677541c2-f981-49e0-a68a-115a74e47284",  
  
    "previousStatus": "PENDENTE",  
  
    "newStatus": "CONFIRMADO",  
  
    "changedBy": "677541c2-f981-49e0-a68a-115a74e47284"  
  
  }  
}
```

2. Demonstração de Comunicação Assíncrona

Para demonstrar a comunicação assíncrona, a API e o worker foram rodados em terminais separados, como dois processos independentes. Quando o cliente faz um **POST /appointments**, a API salva o agendamento no banco de dados, publica o evento no RabbitMQ e já retorna a resposta **201 Created** — sem esperar nenhuma resposta do worker. O worker, rodando no outro terminal, pega a mensagem da fila e processa a notificação por conta própria, sem que a API precise saber que isso aconteceu.

O mesmo acontece nas mudanças de status via **PATCH /appointments/:id/status**: a API muda o status, publica o evento e responde imediatamente. O worker consome e simula o envio da notificação para o cliente ou prestador afetado.

Log do worker processando os eventos

```
> backend@1.0.0 worker
```

```
> ts-node-dev --respawn --transpile-only src/worker.ts
```

```
[INFO] 20:06:48 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 6.0.3)
```

```
[Worker] Iniciando PointDog Event Worker...
```

```
[Consumer] Fila "pointdog.notifications" pronta. Binding: "appointment.*" → "pointdog.events"
```

```
[Consumer] Aguardando mensagens...
```

```
[Worker] Pronto para receber eventos.
```

```
[WORKER] ——— Evento recebido —————
```

```
[WORKER] Tipo: appointment.created
```

```
[WORKER] Timestamp: 2026-05-24T23:08:22.231Z
```

```
[WORKER] Novo agendamento criado: 316006f2-a675-4f87-b390-614a93554ebd
```

```
[WORKER] Cliente: 69636b0b-5aab-43ad-b15c-66d935e961b5 | Pet:  
630ed27c-5d6b-4d4d-8f18-3a153da18ddf
```

```
[WORKER] Prestador: 677541c2-f981-49e0-a68a-115a74e47284 | Serviço:  
af7f74e3-1666-44b4-acdb-979f7f0b1bf4
```

```
[WORKER] Agendado para: 2027-01-15T10:00:00.000Z
```

```
[WORKER] → Simulando notificação ao prestador 677541c2-f981-49e0-a68a-115a74e47284...
```

```
[WORKER] ✓ Notificação enviada.
```

```
[WORKER] —————
```

```
[WORKER] ——— Evento recebido —————
```

```
[WORKER] Tipo: appointment.status_changed
```

```
[WORKER] Timestamp: 2026-05-24T23:08:44.115Z
```

```
[WORKER] Agendamento: 316006f2-a675-4f87-b390-614a93554ebd
```

```
[WORKER] Status: PENDENTE → CONFIRMADO
```

[WORKER] Alterado por: 677541c2-f981-49e0-a68a-115a74e47284

[WORKER] → Simulando notificação ao cliente 69636b0b-5aab-43ad-b15c-66d935e961b5...

[WORKER] ✓ Notificação enviada.

[WORKER] _____

[WORKER] — Evento recebido — _____

[WORKER] Tipo: appointment.status_changed

[WORKER] Timestamp: 2026-05-24T23:08:44.222Z

[WORKER] Agendamento: 316006f2-a675-4f87-b390-614a93554ebd

[WORKER] Status: CONFIRMADO → CONCLUIDO

[WORKER] Alterado por: 677541c2-f981-49e0-a68a-115a74e47284

[WORKER] → Simulando notificação ao cliente 69636b0b-5aab-43ad-b15c-66d935e961b5...

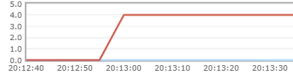
[WORKER] ✓ Notificação enviada.

[WORKER] _____

Overview

Totals

Queued messages last minute ?



Ready	4
Unacked	0
Total	4

Message rates last minute ?



Publish	0.00/s	Deliver (auto ack)	0.00/s	Get (manual ack)	0.00/s	Unroutable (return)	0.00/s
Publisher confirm	0.00/s	Consumer ack	0.00/s	Get (auto ack)	0.00/s	Unroutable (drop)	0.00/s
Deliver (manual ack)	0.00/s	Redelivered	0.00/s	Get (empty)	0.00/s	Disk read	0.00/s
						Disk write	0.00/s

Global counts ?

Connections: 1 Channels: 1 Exchanges: 8 Queues: 1 Consumers: 0

Nodes

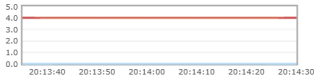
Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats	+/-
rabbit@6f03a6adfc1e	46 1048576 available	1 943629 available	465 1048576 available	174 MiB 3.0 GiB high watermark	942 GiB 48 MiB low watermark	8m 6s	12	basic 2 rss	This node All nodes	

Churn statistics

Queue poindog.notifications

Overview

Queued messages last minute ?



Ready	4
Unacked	0
Total	4

Message rates last minute ?



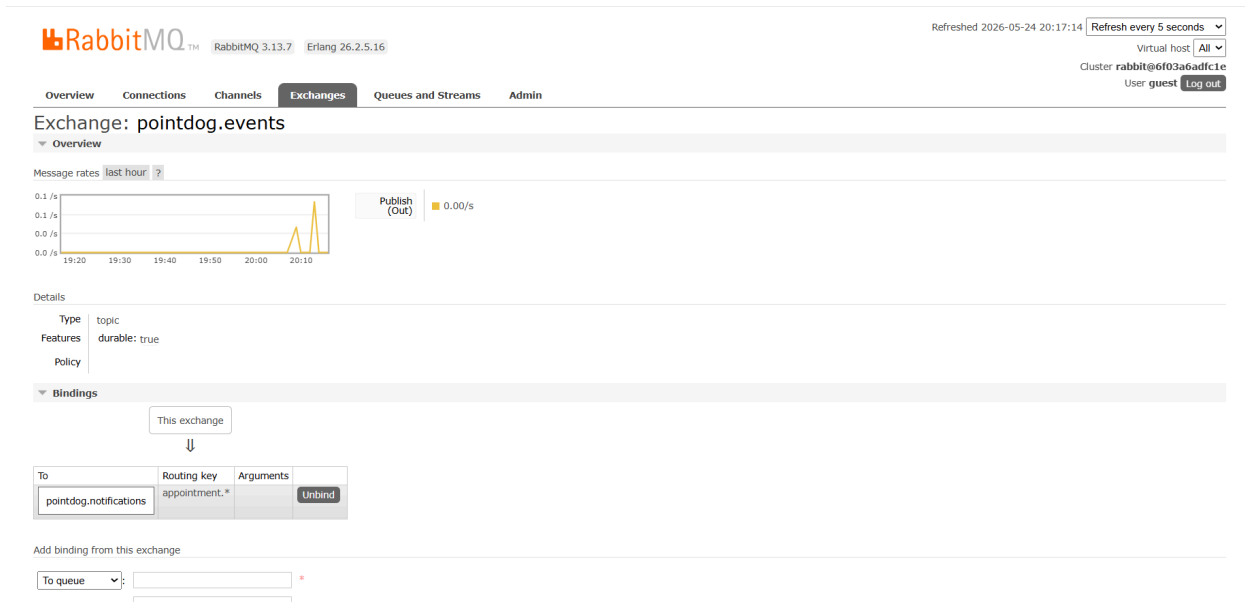
Publish	0.00/s	Consumer ack	0.00/s	Get (auto ack)	0.00/s
Deliver (manual ack)	0.00/s	Redelivered	0.00/s	Get (empty)	0.00/s
Deliver (auto ack)	0.00/s	Get (manual ack)	0.00/s		

Details

Features	State	Messages ?	Total	Ready	Unacked	In memory	Persistent	Transient, Paged Out
queue storage version: 1	idle	0	4	4	0	1	4	0
Policy	Consumers	0	1.5 KiB	1.5 KiB	0 B	402 B	1.5 KiB	0 B
Operator policy	Consumer capacity ?	0%	12 KiB					
Effective policy definition								

Consumers (0)

Bindings (2)



3. Relatório de Integração

Escolha da Ferramenta

Para esta sprint foi utilizado o **RabbitMQ** como middleware de mensageria. A escolha foi baseada em alguns pontos práticos:

- É simples de subir usando Docker Compose, sem precisar configurar nada extra (ao contrário do Kafka, que depende do ZooKeeper).
- Vem com um painel web (Management UI) que facilita muito na hora de testar, já que dá para ver as filas, as mensagens e o exchange direto no navegador.
- A biblioteca **amqp-lib** usada no Node.js é bem documentada e cobre bem o protocolo AMQP 0-9-1.

Padrão Utilizado

Foi usado o padrão **Topic Exchange**. O produtor publica mensagens com routing keys no formato **appointment.created** ou **appointment.status_changed**, e o worker consome tudo que bate com o padrão **appointment.***. Isso é mais flexível do que um Direct Exchange, porque se no futuro surgir um novo tipo de evento de agendamento, o worker já vai recebê-lo sem precisar mudar nada na configuração da fila.

Integração com a Arquitetura

Para não misturar a lógica de negócio com a infraestrutura de mensageria, foi criada a interface `IEventPublisher` na pasta `shared/messaging/`. Os use cases (`CreateAppointmentUseCase` e `UpdateAppointmentStatusUseCase`) dependem só dessa interface, sem saber se por baixo é RabbitMQ ou qualquer outra coisa. Em produção, o `RabbitMQPublisher` é injetado via construtor no `server.ts`. Nos testes, é usado um `NullEventPublisher` que não faz nada, o que permite rodar os 49 testes sem precisar do Docker rodando.

Desafios Encontrados

O primeiro problema foi que, ao subir tudo via Docker Compose, a API pode iniciar antes do RabbitMQ estar pronto para aceitar conexões. A solução foi deixar o `connect()` bem explícito no `server.ts`, assim fica claro onde a falha acontece se o broker não estiver disponível.

Outro problema foi com a tipagem do `amqplib` no TypeScript: a versão 2.x mudou o tipo do objeto de conexão de `Connection` para `ChannelModel`, o que gerou um erro de compilação. Depois de identificar o problema na documentação da biblioteca, o ajuste foi simples.

Por fim, garantir que os testes continuassem funcionando sem RabbitMQ também foi um ponto de atenção. A solução do `NullEventPublisher` resolveu isso de forma limpa, sem precisar mockar chamadas de rede nem configurar nada especial no ambiente de testes.